SIGUEMA — Prompts por atividade

Projeto: app mobile Flutter da UEMA (CTIC) · Baseado nas decisões e propostas até aqui · Substitua o que estiver em [colchetes]

> **Como usar.** Copie o prompt, troque os [colchetes] e cole no Claude Code (dentro da pasta do projeto). Os prompts citam skills (ex.: "use a skill writing-plans"); quem usa ChatGPT/Codex pode apagar essa linha e colar antes o **Prompt de contexto** (P0). **Nunca** cole CPF, senha, token ou dado real de aluno em um prompt. Stack e arquitetura ainda são **propostas**; os pontos em aberto estão na ADR 0001.

Índice

- **0.** Contexto: P0
- **A. Planejamento:** P1 histórias · P2 tarefas da sprint · P3 plano de funcionalidade · P4 contrato da API
- **B. Setup:** P5 projeto base · P6 arquitetura · P7 CI · P8 Docker do servidor
- **C. Desenvolvimento:** P9 rede · P10 sessão e tokens · P11 estado e rotas · P12 login e vínculo · P13 tela com dados · P14 layout responsivo · P15 Figma para código · P16 cache offline · P17 push
- **D. Qualidade:** P18 TDD · P19 teste de widget · P20 teste de integração · P21 análise estática · P22 erro em execução · P23 bug
- **E. Entrega:** P24 revisão de código · P25 fechar branch e PR · P26 segurança e LGPD · P27 release nas lojas
- **F. Documentação:** P28 ADR · P29 atualizar docs

0. Contexto

P0 · Prompt de contexto (cole no início em qualquer ferramenta)

ChatGPT / Codex · sem skill — Use quando a ferramenta não lê o AGENTS.md sozinha.

```
Você é meu par de programação no projeto SIGUEMA Mobile (app Flutter da UEMA, feito pela CTIC, para Android e iOS).
Contexto: piloto com ~3 mil usuários, meta ~29 mil. Login do SIGUEMA por CPF e senha; depois o usuário escolhe o vínculo (aluno, professor, servidor). Integra com o SIGAA. Hospedagem no servidor da UEMA. Equipe: 2 devs; desenvolvo em Windows com VS Code.
Stack proposta: Riverpod, go_router, dio, freezed + json_serializable, flutter_secure_storage, local_auth, drift (só onde precisar), Sentry. Arquitetura: feature-first + MVVM + Repository; DTO + Mapper para o SIGAA/SIGUEMA.
Regras: nunca guardar senha no aparelho; nunca registrar CPF, senha ou token em log; sem segredos no repositório; dados de teste sempre fictícios; PR pequeno, revisado pelo outro dev; commits no padrão Conventional Commits.
Se faltar uma decisão (ex.: formato da API), pergunte antes de inventar. Responda em português do Brasil.
Minha tarefa agora: [descreva].
```

A. Planejamento

P1 · Escrever user stories

Reunião / refinamento issue: historia.md

```
Transforme as anotações abaixo em user stories no formato "Como [perfil], quero [ação], para [benefício]". Para cada história entregue: critérios de aceitação (Dado/Quando/Então), o que fica fora do escopo, dependências (SIGAA, CTIC, lojas) e uma sugestão de tamanho (P/M/G). Perfis possíveis: aluno, professor, servidor, visitante. Marque quais entram no MVP e quais ficam para depois, e liste as perguntas que ainda preciso fazer ao PO. Não invente regras da UEMA: onde faltar informação, escreva "A CONFIRMAR".
Anotações da reunião:
[cole aqui, sem dados pessoais]
```

P2 · Quebrar histórias em tarefas da Sprint

Sprint Planning issue: tarefa.md

```
Estas são as histórias escolhidas para a Sprint [n], de [2] semanas, com [2] devs: [cole as histórias].
Quebre em tarefas técnicas de no máximo 1 dia cada, no formato do template tarefa.md (objetivo, passos, pronto quando, dependências, estimativa). Separe o que dá para fazer em paralelo entre dois devs sem tocar nos mesmos arquivos. Aponte riscos e tarefas que dependem da CTIC. Ao final, sugira a ordem dos PRs.
```

P3 · Plano de implementação de uma funcionalidade

skill: writing-plans Antes de escrever qualquer código.

```
Use a skill writing-plans. Funcionalidade: [ex.: tela de avisos com lista e detalhe]. Leia o AGENTS.md e a estrutura atual de lib/. Entregue um plano em etapas pequenas e verificáveis, indicando os arquivos a criar ou alterar (UI, ViewModel, Repository, DTO e Mapper), os testes de cada etapa e o que depende de decisão minha. Não escreva código ainda. Salve o plano em docs/plans/[nome].md.
```

P4 · Contrato da API (OpenAPI) e mock

sem skill Permite app e servidor andarem em paralelo.

```
Crie um contrato OpenAPI 3 (arquivo docs/api/openapi.yaml) para as rotas: POST /auth/login (CPF e senha, retorna vínculos e token temporário), POST /auth/vinculo (retorna token definitivo e refresh token), POST /auth/refresh e GET /avisos. Defina esquemas, códigos de erro padronizados (401, 403, 422, 429, 5xx) e exemplos com dados FICTÍCIOS. Em seguida, sugira como subir um mock local desse contrato para o app usar durante o desenvolvimento. Marque como "A CONFIRMAR COM A CTIC" tudo que depender de como o SIGUEMA/SIGAA realmente responde.
```

B. Setup

P5 · Criar o projeto Flutter base com flavors

Tarefa T-15/T-16 skill: dart-run-static-analysis

> Crie o projeto Flutter na raiz deste repositório (nome do pacote: siguema_mobile; organização br.uema.siguema), com Android e iOS. Configure flavors dev, homologacao e producao (entradas main_dev.dart, main_homologacao.dart, main_producao.dart, com a URL base da API por ambiente, sem segredos no código). Adicione flutter_lints com regras estritas, as dependências da stack proposta no AGENTS.md e uma tela inicial vazia. Rode dart format, flutter analyze e flutter test e mostre o resultado real. Não faça commit.

P6 · Estrutura de pastas e arquitetura

skill: flutter-apply-architecture-best-practices Tarefa T-19

> Use a skill flutter-apply-architecture-best-practices. Monte o esqueleto feature-first + MVVM + Repository em lib/: core/ (network, storage, error, config) e features/ (auth, avisos, horarios, notas), cada feature com data, domain e presentation. Crie apenas as pastas e um exemplo mínimo completo na feature auth (interface do repository, implementação, DTO, mapper e ViewModel com Riverpod). Explique em 10 linhas as regras de dependência entre camadas e atualize o AGENTS.md se algo mudar.

P7 · CI no GitHub Actions

Tarefa T-17

> Revise .github/workflows/ci.yml. Garanta que, com o projeto Flutter já criado, ele rode: pub get, build_runner, dart format (falhando se mudar algo), flutter analyze e flutter test, usando cache. Acrescente cobertura de testes como artefato. Mostre o que mudou e o que ainda não consegui validar sem rodar no GitHub.

P8 · Docker do servidor (BFF / homologação)

Tarefa T-18 depende da CTIC

> Crie um docker-compose.yml para o ambiente de homologação com: API [linguagem escolhida], PostgreSQL, Redis e Nginx como proxy reverso com HTTPS. Use variáveis de ambiente por .env (gere um .env.example sem segredos reais), volumes para o banco, healthchecks e reinício automático. Liste o que preciso pedir à CTIC (portas, certificado TLS, DNS, saída para a internet) e como fazer backup do banco. Não invente valores de produção.

C. Desenvolvimento

P9 · Camada de rede (dio)

Tarefa T-20 skill: test-driven-development

```
Use a skill test-driven-development. Implemente em lib/core/network o cliente dio com: URL base por flavor, timeouts, interceptor que adiciona o token de acesso, interceptor que em caso de 401 tenta renovar o token UMA vez e repete a chamada, e conversão de erros para um tipo próprio (sealed class: SemConexao, NaoAutorizado, ErroDeValidacao, ErroDoServidor). Nunca registrar CPF, senha ou token em log. Escreva primeiro os testes com mocktail. Mostre o resultado real de flutter test.
```

P10 · Sessão e tokens

Tarefa T-21 segurança

```
Implemente em lib/core/storage o armazenamento de sessão com flutter_secure_storage: guardar apenas token de acesso e refresh token (jamais a senha), apagar tudo no logout e expor um repositório de sessão testável por interface. Adicione, como opcional atrás de uma flag, o desbloqueio por biometria com local_auth que libera o refresh token já salvo. Escreva os testes com um fake do storage e descreva os riscos restantes (aparelho com root/jailbreak).
```

P11 · Estado e rotas com redirecionamento

skill: flutter-setup-declarative-routing Tarefa T-22

```
Use a skill flutter-setup-declarative-routing. Configure o go_router com Riverpod: rotas /login, /escolher-vinculo e /home; redirecionamento (não logado vai para /login; logado sem vínculo vai para /escolher-vinculo); suporte a deep link para /avisos/:id. Escreva testes das regras de redirecionamento sem depender de tela. Mostre o resultado de flutter analyze e flutter test.
```

P12 · Login por CPF e escolha de vínculo

Tarefas T-24/T-25 skills: test-driven-development, flutter-add-widget-test

```
Use as skills test-driven-development e flutter-add-widget-test. Implemente a feature auth: tela de login (CPF com máscara e validação, senha com opção de mostrar/ocultar, estados de carregando e erro), ViewModel com AsyncValue, AuthRepository usando o contrato de docs/api/openapi.yaml (contra o mock), e a tela de escolha de vínculo que só aparece quando há mais de um. Acessibilidade: rótulos Semantics e foco correto. Use apenas CPFs fictícios nos testes. Não registre o CPF em log.
```

P13 · Uma tela de dados (ex.: avisos)

Tarefa T-26

```
Siga o plano em docs/plans/[nome].md (ou use a skill writing-plans antes). Implemente a feature avisos: lista com carregando, vazio e erro, tela de detalhe, pull-to-refresh, DTO + Mapper a partir do contrato da API, e testes de ViewModel, Repository e widget. Mantenha o padrão da feature auth. Mostre o resultado real dos testes.
```

P14 · Layout responsivo

skill: flutter-build-responsive-layout

```
Use a skill flutter-build-responsive-layout. Ajuste a tela [nome] para funcionar em celular pequeno, celular grande e tablet, em retrato e paisagem, e com fonte do sistema aumentada (acessibilidade). Use LayoutBuilder/MediaQuery e evite tamanhos fixos. Escreva testes de widget para pelo menos duas larguras e informe como validei.
```

P15 · Do Figma para código

skill: figma | exige Figma MCP conectado

```
Use a skill figma. Link do frame: [url do Figma]. Busque o contexto e a captura do design, reaproveite os tokens (cores, tipografia, espaçamento) em um tema central do app e implemente a tela em Flutter seguindo a arquitetura do projeto. Se o Figma MCP não estiver conectado, pare e me avise. Ao final, compare o resultado com a captura e liste as diferenças.
```

P16 · Cache offline (somente onde precisar)

Pergunta Q46

```
Implemente cache offline com drift apenas para [telas definidas, ex.: horários da semana]. Padrão cache-aside no Repository: tenta a rede, salva no banco e, sem conexão, devolve o cache indicando na UI que o dado está desatualizado e a data da última atualização. Migrações versionadas, testes do Repository com banco em memória. Não guarde dados sensíveis sem criptografia; liste o que foi guardado.
```

P17 · Notificações push

depende da decisão sobre Firebase

```
Implemente o registro do token de push (FCM) e o tratamento de mensagens com o app aberto, em segundo plano e fechado, abrindo a rota correta por deep link. O texto do push é sempre genérico ("Você tem um novo aviso"); o conteúdo real é buscado na API ao abrir. Registre o token no servidor apenas depois do login e remova no logout. Liste o que precisa ser configurado fora do código (APNs, google-services.json, que não vai para o Git).
```

D. Qualidade

P18 · Desenvolver com TDD

skill: test-driven-development

```
Use a skill test-driven-development. Regra a implementar: [descreva a regra]. Escreva primeiro o teste que falha, mostre-o falhando, implemente o mínimo para passar, refatore e rode a suíte inteira. Relate cada passo com a saída real dos testes.
```

P19 · Teste de widget

skill: flutter-add-widget-test

```
Use a skill flutter-add-widget-test. Crie testes de widget para [tela/componente], cobrindo: renderização com dados, estado de carregando, estado de erro, interação (toque, digitação, rolagem) e acessibilidade (Semantics). Use fakes dos repositórios via override de providers. Rode e mostre o resultado real.
```

P20 · Teste de integração (fluxo completo)

skill: flutter-add-integration-test

```
Use a skill flutter-add-integration-test. Crie um teste de integração do fluxo: abrir o app, entrar com CPF fictício, escolher o vínculo e ver a tela inicial, usando o servidor mock. Explique como rodar em emulador Android e o que falta para rodar em iOS (precisa de Mac).
```

P21 · Análise estática antes de commitar

skill: dart-run-static-analysis

```
Use a skill dart-run-static-analysis. Rode dart analyze e dart fix --apply, mostre o que foi corrigido automaticamente e liste o que sobrou para eu decidir. Não altere comportamento nem silencie regras com "ignore" sem me avisar.
```

P22 · Erro em execução (app rodando)

skill: dart-fix-runtime-errors

```
Use a skill dart-fix-runtime-errors. O app está rodando em [emulador/aparelho] e dá erro ao [ação]. Busque o stack trace ativo, localize a linha, explique a causa, aplique a correção mínima e verifique com hot reload. Se a causa for dado inesperado da API, proponha também um teste que reproduza.
```

P23 · Corrigir um bug a partir de uma issue

issue: bug.md skill: test-driven-development

```
Corrija o bug da issue #[n]: [cole título e passos, sem dados pessoais]. Primeiro reproduza com um teste que falha, depois corrija a causa (não o sintoma), rode a suíte completa e descreva o que mudou. Crie a branch fix/[nome-curto] e prepare a mensagem de commit em Conventional Commits. Não faça push.
```

E. Entrega

P24 · Pedir revisão de código

skill: requesting-code-review

```
Use a skill requesting-code-review. Revise as mudanças da branch atual contra a main: bugs, violações da arquitetura do AGENTS.md, dado sensível em log, falta de testes e itens da Definition of Done do template de PR. Liste os achados por gravidade, com arquivo e linha, e diga o que é opinião e o que é erro.
```

P25 · Fechar a branch e abrir o PR

skill: finishing-a-development-branch

```
Use a skill finishing-a-development-branch. Confirme que format, analyze e test passam, organize os commits em Conventional Commits e escreva a descrição do PR seguindo .github/pull_request_template.md, incluindo "Closes #[n]", como testar e riscos para o revisor. Mostre o texto e os comandos para eu conferir antes. Só faça push se eu mandar.
```

P26 · Revisão de segurança e LGPD

antes de cada release

```
Faça uma revisão de segurança e LGPD do código atual: dados sensíveis em logs, prints ou analytics; armazenamento de tokens; senha nunca persistida; HTTPS e validação de certificado; permissões do Android e do iOS; segredos versionados por engano (procure .env, chaves, google-services.json); conteúdo de push; dependências desatualizadas ou com vulnerabilidade conhecida. Use o OWASP Mobile Top 10 como guia. Entregue uma tabela: achado, gravidade, arquivo, correção sugerida.
```

P27 · Preparar release para as lojas

depende de Mac e contas das lojas

```
Prepare o checklist e os arquivos para publicar a versão [x.y.z] na Play Store e na App Store: versão e build number, assinatura (sem versionar chaves), ícones e splash, política de privacidade e declaração de dados (LGPD), permissões justificadas, textos e capturas de tela, e um fastlane de exemplo. Aponte o que depende de conta da UEMA (D-U-N-S, conta de organização) e do Mac para o iOS.
```

F. Documentação

P28 · Registrar uma decisão (ADR)

docs/adr/

```
Escreva a ADR 000[n] em docs/adr/ no formato da 0001 (contexto, decisão, alternativas, consequências, status) para a decisão: [ex.: reaproveitar a API do SIGUEMA no MVP]. Use as respostas da CTIC que informarei abaixo e marque como "A CONFIRMAR" o que não tiver sido dito. Respostas: [cole].
```

P29 · Atualizar a documentação

fim de sprint

```
Compare o código atual com README.md, AGENTS.md e CONTRIBUTING.md. Liste o que ficou desatualizado (comandos, estrutura de pastas, decisões) e proponha as edições mínimas. Não reescreva o que está correto.
```

Lembrete final. Sempre peça para mostrar a saída real de `flutter analyze` e `flutter test`, revise o diff antes de aceitar e só faça commit e push quando você decidir. Código gerado por IA entra no PR como código de qualquer colega: passa por revisão.